KadeeStudio — Architecture & Implementation Plan

Oct 31, 2026 · @Duggie

Vision and scope

KadeeStudio is one native Rust desktop suite, with a self-hosted collaboration server, that replaces separate tools for Markdown, PDF, conversion, dashboards and reports, and hosts the ArtCraft Crafting Apps. The first deliverable is only the start screen: a shell with module cards, recent files and settings, no editors and no AI. AI follows in Phase 1, before the editors.

Module	Replaces	First release
Kadee AI	Separate chat tools; one assistant that works across every module and file	Phase 1
Kadee Markdown	Typora, MarkText, Word for plain-text docs	Phase 2
Kadee PDF	Acrobat (view, annotate, organise, sign)	Phase 4
Kadee Convert	Pandoc-style converters, online PDF tools (reached from File › Convert and the right-click menu; no start-screen tile)	Phase 5
Kadee Dashboards	Metabase, Power BI dashboards	Phase 6
Kadee Reports	Power BI paginated reports, Crystal-style reports	Phase 6
Crafting Apps	CadCraft, DeckCraft, DesignCraft, EffectCraft, FigCraft, FilmCraft, GridCraft, LightCraft, PdfCraft, PhotoCraft, SolidCraft, SoundCraft, VectorCraft and WordCraft, launched or embedded	Phase 7

Design principles:

- **Native Rust end to end.** No Electron, no JavaScript in the app shell; one codebase for macOS, Windows and Linux, with a WASM build later.
- **Everything is a command.** Every button, shortcut and menu item calls a command with an id (`file.new.markdown`, `home.open`), as in mdcraft and the Crafting Apps. This is what lets the Phase 1 AI drive every action, and later the CLI and MCP too.

- **Local-first, collaborative by default.** Documents are CRDTs that work offline and sync to a KadeeStudio server on-prem or in the cloud.
- **Modules are crates.** The shell knows modules only through a small trait, so each editor can be built, tested and shipped on its own.
- **Open formats.** Markdown stays plain Markdown, PDF stays PDF, dashboards are stored as readable JSON/YAML definitions.

What we reuse

KadeeStudio borrows its shell layout from GenOffice, its Markdown engine from mdcraft, its collaboration stack from SolarOffice, and its native UI style and engines from the ArtCraft Crafting Apps.

Source	What it is	What KadeeStudio takes
mdcraft	Your Rust Markdown word processor: tao + wry webview, CodeMirror editor, comrak renderer (MarkDown Live) with Mermaid, KaTeX, charts, slides, 11 document and 8 deck templates, CLI and MCP server	`mdcraft-core` renderer and templates as library crates; the ribbon layout, File view and command ids; its MCP server as a model for the AI tools
SolarOffice	Your earlier Rust suite: Dioxus 0.7 + Tailwind, home screen with tab strip, sidebar (Recent, Starred, folders), quick-start cards, sortable file list; an AI layer (`solar-ai`: providers, per-module models, Ask AI chat, AI side pane); planned Yrs CRDT + Axum/PostgreSQL sync server	The home screen feature list and `home.json` library model; the AI provider and model settings for Phase 1; the Yrs + Axum + PostgreSQL collaboration design
GenOffice	Open-source AI office suite (Apache-2.0): Electron shell hosting Docs, Sheets, Slides, PDF and Markdown in tabs, light/dark/system theme on shared tokens, an AI panel in every app	The start-screen pattern: home tab plus one tab per open file, a module launcher, design tokens; the idea of one AI panel shared by all modules. No code (TypeScript/Electron)

Source	What it is	What KadeeStudio takes
ArtCraft Crafting Apps	Pure-Rust apps (MIT OR Apache-2.0): CadCraft, DeckCraft, DesignCraft, EffectCraft, FigCraft, FilmCraft, GridCraft, LightCraft, PdfCraft, PhotoCraft, SoundCraft, VectorCraft, WordCraft; egui frontends kept in their own crate, separate from the engine crates	egui as the UI toolkit; engine crates (e.g. PDF, vector, grid) as dependencies; the craft-launcher approach to install and open apps; their file types in the start-screen document list
SolidCraft	Open-source parametric 3D CAD in pure Rust (a clean-room Fusion 360), from Tangency Labs rather than the ArtCraft team; STEP, IGES, 3MF, STL, OBJ, glTF and URDF export; MCP server for agents	Listed with the Crafting Apps; its 3D file types in the document list; launch and file hand-off in Phase 7

Two cautions. The ArtCraft and SolidCraft names and logos are not covered by the code licences, so KadeeStudio uses its own branding and only lists the apps by name. mdcraft's editor and preview run JavaScript inside a webview, so the Markdown module needs a native rewrite of the editor surface (see Technology choices).

Technology choices

The UI is egui/eframe on wgpu, chosen over Dioxus and Tauri because it is pure Rust with no webview, matches the ArtCraft apps so their crates drop in, and compiles to WASM for a later browser client.

Layer	Choice	Why
UI toolkit	egui + eframe (wgpu renderer, glow fallback)	Immediate-mode, fully native, same toolkit as the Crafting Apps; one codebase for desktop and web

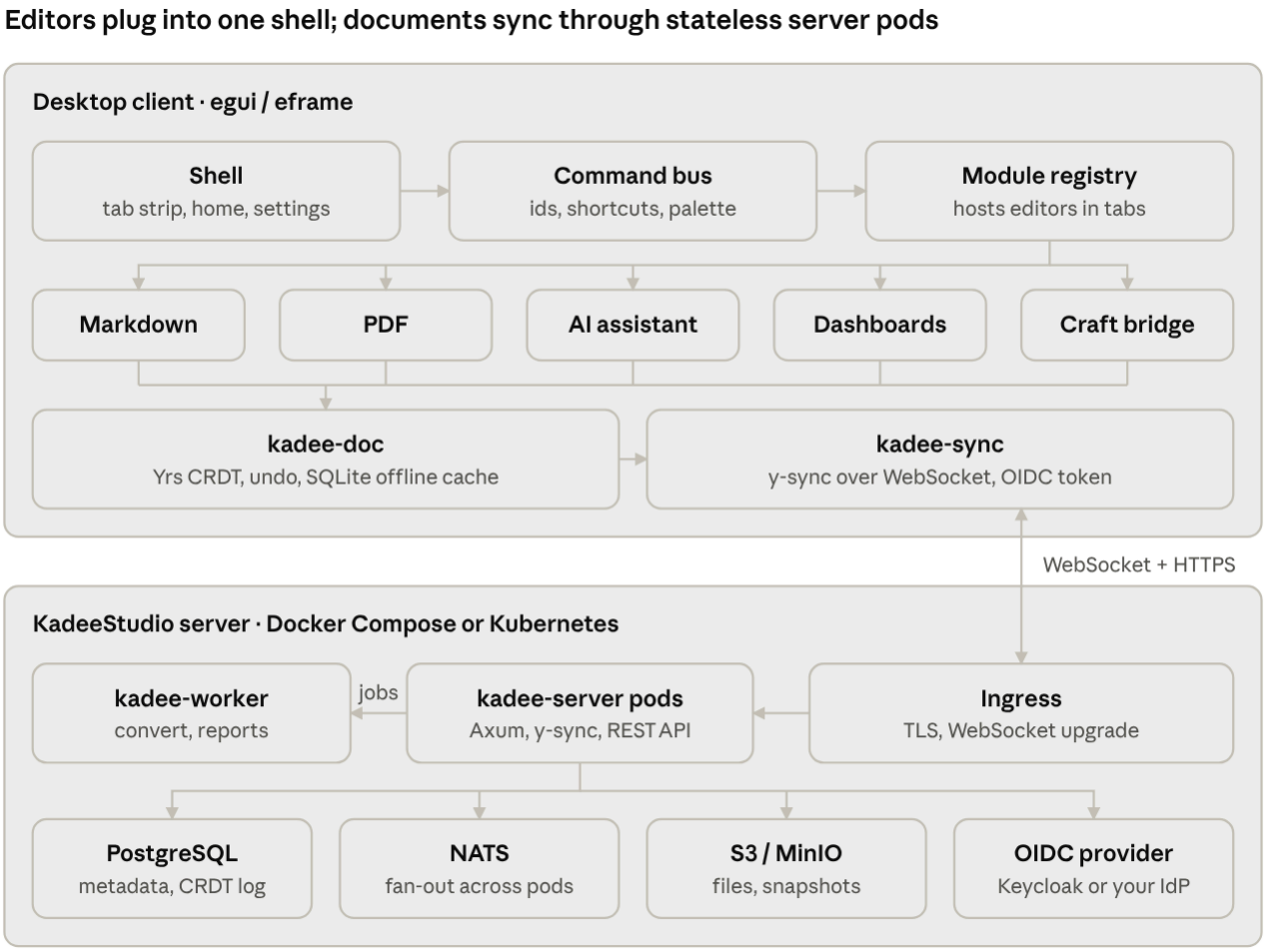
Layer	Choice	Why
AI	`kadee-ai` crate, built from SolarOffice's `solar-ai`: Anthropic, OpenAI, Azure OpenAI, Mistral, and local models through Ollama or any OpenAI-compatible server; tool calls mapped to KadeeStudio commands; `rmcp` for an MCP server	Works with cloud keys or fully on-prem; the assistant acts through the same commands as the user
Text and fonts	egui text layout now; cosmic-text / parley for the Markdown editor surface	Rich editing with bidi, shaping and system fonts
Markdown	comrak (GFM) via `mdcraft-core`	Already proven in mdcraft
Math and diagrams	Typst for math and layout; native Mermaid-subset renderer later	Removes the JavaScript dependency of KaTeX/Mermaid
PDF read and render	pdfium-render (behind a feature flag) or the PdfCraft engine	Fast, accurate rendering
PDF write and convert	Typst and krilla for generation, lopdf for page operations	Markdown to PDF and report export in pure Rust
Data and query	Apache DataFusion + Arrow; sqlx connectors (PostgreSQL, MySQL, MS SQL, SQLite); CSV, Parquet, Excel via calamine	A Metabase-style query engine embedded in the app and in the server
Charts	Custom egui chart widgets over Arrow batches; egui_plot for early versions	Native dashboards with no browser
Collaboration	Yrs (Rust port of Y.js CRDT)	Offline editing, conflict-free merges, same model on client and server
Server	Axum + Tokio, WebSocket y-sync protocol, PostgreSQL, S3-compatible object storage, NATS for fan-out	Stateless pods that scale on Kubernetes
Identity	OIDC (Keycloak on-prem; Entra ID, Google, Okta in the cloud)	One login flow for every deployment

Layer	Choice	Why
Local storage	`~/Documents/KadeeStudio` for files; config dir for `home.json` and `ai.toml`; OS keychain for API keys; SQLite for the local CRDT cache	Works fully offline; no keys in plain files
Build and release	Cargo workspace, `cargo xtask`, cargo-dist or cargo-packager for .dmg / .msi / .deb / .rpm / AppImage; GitHub Actions	Same release pattern as the Crafting Apps

The Markdown editor is the hardest UI piece in pure egui. The plan keeps an optional wry preview pane behind a feature flag, so rich previews from mdcraft (Mermaid, Vega, Plotly) work on day one while the native renderer catches up.

Architecture

The client is a thin egui shell over a module registry and a command bus; every editor is a crate that implements one trait, and every document lives in a Yrs CRDT that syncs through the collaboration server.



KadeeStudio architecture · client and server

The shell routes every action through the command bus to whichever module owns the tab; modules keep their documents in kadee-doc, and kadee-sync carries the changes to the server, which stores them and relays them to other pods.

Cargo workspace (the start screen ships the crates marked Phase 0):

Crate	Job	Phase
`apps/kadeestudio`	Desktop binary: window, tab strip, home screen, settings	0
`crates/kadee-core`	Module catalogue (`ModuleKind`), Crafting Apps catalogue, file-type registry (which module or Crafting App opens each extension), recent/starred library, settings, `home.json` persistence	0
`crates/kadee-ui`	Design tokens, light/dark themes, icons, shared widgets (cards, pills, file table)	0

Crate	Job	Phase
`crates/kadee-commands`	Command ids, registry, keyboard bindings, command palette	0 (minimal)
`crates/kadee-ai`	Providers and models, Ask AI chat, the AI side pane, document context extraction, tool calls onto commands, MCP server	1
`crates/kadee-module-api`	`Module` trait the shell uses to host an editor in a tab	2
`crates/kadee-doc`	Yrs document wrapper, local SQLite cache, undo, awareness (cursors)	2
`crates/kadee-markdown`	Markdown editor module on `mdcraft-core`	2
`crates/kadee-sync`	Client side of the y-sync WebSocket protocol, auth tokens, offline queue	3
`server/kadee-server`	Axum collaboration and API server, AI gateway for organisation-wide keys and policy	3
`crates/kadee-pdf`	PDF viewer/editor module	4
`crates/kadee-convert`	Format conversion pipeline (MD, PDF, HTML, DOCX, PPTX)	5
`server/kadee-worker`	Background jobs: conversion, report rendering, scheduled refresh	5
`crates/kadee-data`	DataFusion query engine, connectors, semantic models	6
`crates/kadee-dash`	Dashboard and report designer modules	6
`crates/kadee-craft`	Bridge to the Crafting Apps (ArtCraft and SolidCraft): launch, embed, file hand-off	7

The module contract every editor implements:

```
pub trait Module: Send {
    fn kind(&self) -> ModuleKind;              // Markdown, Pdf, Dashboard …
    fn title(&self) -> String;                 // tab label
    fn ui(&mut self, ui: &mut egui::Ui, cx: &mut ShellContext);
    fn commands(&self) -> &[CommandSpec];      // ribbon + palette entries
    fn save(&mut self) -> Result<(), ModuleError>;
    fn is_dirty(&self) -> bool;
}
```

Rules that keep the codebase healthy: engine crates never depend on egui (the same rule the Crafting Apps follow), so the server and CLI can reuse them; modules talk to each other only through commands and file hand-off; and every user action goes through `kadee-commands`, never a direct callback.

Collaboration and deployment

The same container images run on a laptop with Docker Compose, on-prem Kubernetes and managed cloud Kubernetes; only the Helm values change.

How a shared edit flows:

1. The client edits its local Yrs document and stores the update in its SQLite cache, so work continues offline.
2. `kadee-sync` sends the update over a WebSocket to any `kadee-server` pod (y-sync protocol, OIDC bearer token).
3. The pod checks the user's role on the document, appends the update to PostgreSQL, and publishes it on a NATS subject per document.
4. Every pod with clients on that document relays the update; cursors and presence travel the same way as Yrs awareness messages.
5. A compaction job merges old updates into a snapshot and stores large binaries (images, PDFs, datasets) in S3-compatible storage.

Component	Image / service	Docker Compose	Kubernetes
API + sync	`kadee-server` (Rust, distroless)	1 container	Deployment, HPA on connections and CPU, PodDisruptionBudget
Workers	`kadee-worker`	1 container	Deployment or KEDA-scaled jobs
Metadata + CRDT log	PostgreSQL 16+	container with volume	CloudNativePG operator on-prem; RDS / Azure Database / Cloud SQL in the cloud
Files and snapshots	MinIO or any S3 API	container with volume	MinIO operator on-prem; S3 / Azure Blob / GCS
Fan-out between pods	NATS	container	NATS Helm chart (3 replicas)

Component	Image / service	Docker Compose	Kubernetes
Identity	Keycloak (optional)	container	Keycloak operator, or an external IdP via OIDC
Ingress	Caddy	container with automatic TLS	ingress-nginx or Gateway API, cert-manager
Observability	OpenTelemetry → Prometheus, Grafana, Loki	optional profile	kube-prometheus-stack

Deliverables in the repo: `deploy/compose/docker-compose.yml` (single-node, `docker compose up`), `deploy/helm/kadeestudio` (chart with values for on-prem and cloud), and multi-arch images (amd64, arm64) published to GHCR. Access control is role-based per workspace and per document (owner, editor, commenter, viewer); audit events go to PostgreSQL.

Module roadmap

Each module ships when it can replace its office counterpart for everyday work, not when it matches every feature.

Kadee AI (Phase 1, before the editors). One assistant for the whole suite, built on SolarOffice's AI layer: provider and model settings (cloud keys or local models), an **Ask AI** chat in the sidebar that answers from starred or selected documents of any type in the list, and an AI pane in every file tab. The assistant acts only through commands (create, open, star, convert, summarise), so whatever it does the user can also do and undo. An MCP server exposes the same commands to Claude Code and other agents. When the server arrives, an AI gateway adds organisation keys, usage limits and audit.

Kadee Markdown (first editor, Phase 2). Port mdcraft: ribbon (Home, Insert, Design, Slides, Review, View), write / split / read modes, templates, slide decks with presenter mode, export to PDF and HTML. New in KadeeStudio: a native egui editor surface, live co-editing with cursors, comments anchored to text, and version history from the CRDT log.

Kadee PDF. View, search, annotate, fill forms, sign; organise pages (merge, split, rotate, reorder); redact; protect with passwords. Reuse the PdfCraft engine where it fits; collaborative annotations stored as a CRDT layer beside the untouched PDF.

Kadee Convert. One pipeline with a shared intermediate model: Markdown ↔ HTML, Markdown → PDF (Typst), DOCX / PPTX / XLSX → Markdown or PDF, images → PDF, OCR for scans. It has no start-screen tile: it is reached from File › Convert, the right-click menu

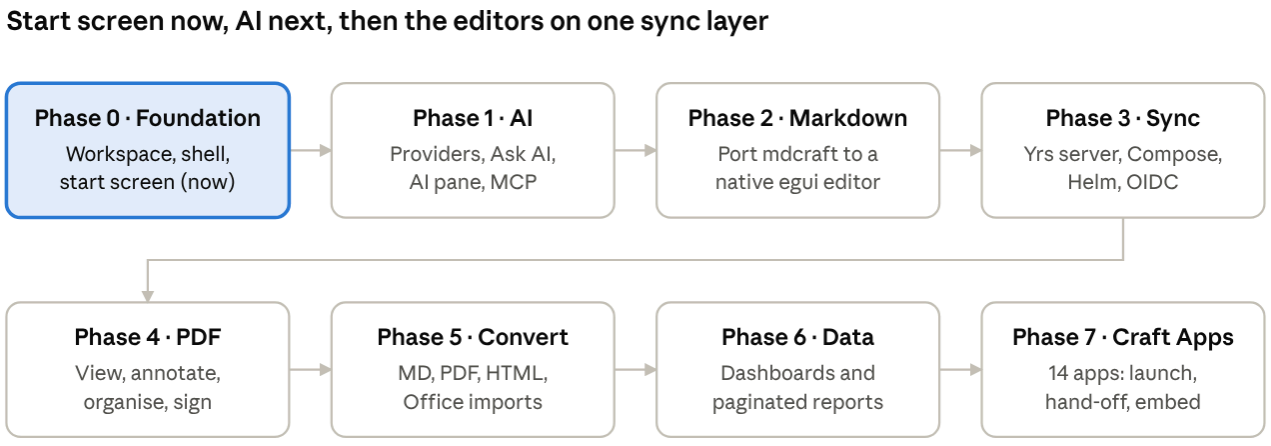
in the document list, and the AI. Runs in the app for single files and in `kadee-worker` for batches.

Kadee Dashboards and Reports. Metabase-style: connect a data source, explore with a visual query builder or SQL, save questions, assemble dashboards with filters and cross-filtering, schedule refresh and e-mail delivery. Power BI-style: a semantic model (measures, relationships), paginated reports exported to PDF. Queries run in DataFusion on the desktop for files and on the server for shared databases, with row-level security.

Crafting Apps. Fourteen apps, listed alphabetically: thirteen ArtCraft apps plus SolidCraft (3D CAD, from Tangency Labs). Their file types are part of the start screen from Phase 0, so **All** lists every document KadeeStudio or a Crafting App can open (see the file-type table in Phase 0). Three levels of integration, in order: (1) a launcher card that installs and opens each app, like ArtCraft's craft-launcher; (2) file hand-off, so a picture in a Markdown document opens in PhotoCraft or a STEP file in SolidCraft and comes back saved; (3) embedding engine crates (vector, grid, deck) as KadeeStudio modules where their licence and APIs allow.

Phased implementation plan

Phase 0 is the start screen delivered with this plan; AI comes next (Phase 1), so every editor after it is built with the assistant in place. Phase 3 (sync server) can run in parallel with Phase 2 because the Markdown module is built on `kadee-doc` from day one.



phased roadmap · 8 phases

Each phase ends with a gate: it ships when a real user can do their everyday work in that module, with tests and packaged builds for macOS, Windows and Linux.

Phase 0 — start screen (delivered now). Scope, modelled on GenOffice's home tab and your SolarOffice home screen:

- Tab strip with a Home tab, one tab per opened file, a **+** new-file menu and a light/dark/system theme switch
- Sidebar: Home, Recent, Starred, Crafting Apps, the KadeeStudio documents folder, Settings
- Greeting and quick-start tiles for Markdown, PDF, Dashboard and Report, plus Open file (no Convert tile)
- A Crafting Apps row with all fourteen apps in alphabetical order, SolidCraft included (cards only, marked Phase 7)
- File list: type filter pills (All, Office, Markdown, PDF, Dashboard, Report, Graphics, Media, CAD), search, sortable Name / Opened / Modified / Size, star toggle. **All** shows every file type that KadeeStudio or any Crafting App opens (table below); each row says which app opens it
- Settings: display name, theme, documents folder; state saved to `home.json` in the config folder (`KADEE_CONFIG_DIR` overrides)
- A file tab shows the file's details, the app that will open it, and an *Open in default app* button until the editors exist
- No AI, no editors, no server

Phase 0 is done when `cargo run -p kadeestudio` opens the start screen on all three desktops, `cargo test` passes for `kadee-core`, and `cargo clippy` is clean.

Document types on the start screen (one registry in `kadee-core`; the first matching row wins, so PDF opens in Kadee PDF rather than PdfCraft):

Filter pill	Extensions	Opens in
Office	.pptx	DeckCraft
Office	.xlsx .csv	GridCraft
Office	.docx	WordCraft
Markdown	.md .markdown .mdd .mds	Kadee Markdown
PDF	.pdf	Kadee PDF
Dashboard	.kdash	Kadee Dashboards
Report	.kreport	Kadee Reports
Graphics	.designcraft	DesignCraft
Graphics	.figcraft .fig	FigCraft
Graphics	.psd .png .jpg .jpeg .webp .tif .tiff	PhotoCraft

Filter pill	Extensions	Opens in
Graphics	.svg .eps .emf .wmf .vectorcraft	VectorCraft
Media	.effectcraft	EffectCraft
Media	.mp4 .mov .mkv .webm	FilmCraft
Media	.dng .cr2 .cr3 .nef .arw .raf .orf .rw2	LightCraft
Media	.wav .aif .aiff .flac .mp3 .ogg .m4a .caf .mid .midi	SoundCraft
CAD	.dwg .dxf	CadCraft
CAD	.step .stp .iges .igs .3mf .stl .obj .gltf .glb	SolidCraft

The `.designcraft` and `.effectcraft` project extensions follow VectorCraft's `.vectorcraft` pattern and still need confirming against those apps; the list is checked against each app's documentation before Phase 7.

Phase 1 — AI. Scope:

- Settings › AI providers (add, test connection, local servers without a key) and AI models (a default plus one per module); API keys in the OS keychain
- **Ask AI** in the sidebar: a streaming chat that can use starred or selected documents; text is extracted from Markdown, PDF and Office files; token and cost footer, Stop, Clear
- An AI pane in every file tab, scoped to that file
- Tools are commands: the assistant can search, open, create, star and summarise documents; anything that writes or deletes asks first
- An MCP server (`kadeestudio mcp`) exposing the same commands to agents

Phase 1 is done when Ask AI answers from a starred PDF and a Markdown file with both a cloud model and a local model, and every action the assistant takes appears in the command log.

Risks and open decisions

Risk	Impact	Mitigation
AI arrives before the editors, so it has little to act on	Phase 1 feels thin	Scope it to what exists: Ask AI over documents of every type, summaries, search, creating and organising files; the editor tools plug in as each module ships
Documents sent to cloud AI providers	Data leaves the organisation	Local models via Ollama or an on-prem OpenAI-compatible server; per-workspace provider policy; the server's AI gateway logs every call
A rich Markdown editor in pure egui is a large build	Phase 2 slips	Start with egui's multiline editor plus syntax colouring; keep the wry preview pane behind a feature flag; move to a cosmic-text surface once it is stable
Mermaid, Vega, Plotly have no Rust renderers	Some mdcraft previews missing natively	Keep them in the optional webview pane; render the common subset natively; Typst for math
CRDT history grows without bound	Slow loads, large database	Snapshot and compaction job; keep N days of fine-grained history
Office-format fidelity (DOCX, PPTX)	Conversion complaints	Convert to Markdown/PDF only at first; borrow WordCraft, DeckCraft and GridCraft parsers later
Crafting Apps (ArtCraft and SolidCraft) evolve independently	Embedded crates break; file-type list drifts	Pin versions; start with launch and file hand-off, embed only stable engine crates; re-check the file-type registry each release

Risk	Impact	Mitigation
Dashboards scope creep toward full Power BI	Phase 6 never ends	Ship Metabase parity first (questions, dashboards, filters, schedules), then semantic models

Open decisions for you:

- [ ] Confirm egui/eframe over Dioxus (SolarOffice) for the shell
- [ ] Licence for KadeeStudio itself (MIT OR Apache-2.0 matches mdcraft and the Crafting Apps)
- [ ] Identity provider for on-prem: bundled Keycloak or customer IdP only
- [ ] AI providers to support at launch, and whether a local model is the default
- [ ] Repository name and GitHub location (for example `kdugardyn66/kadeestudio`)

Sources

- mdcraft repository
- SolarOffice repository
- GenOffice repository
- ArtCraft craft-launcher
- VectorCraft (egui frontend in its own crate)
- SolidCraft and an overview of it